

INF220 / Unidad 1 / Práctica 1

PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

Práctica 1 — Fundamentos de POO en Java

Cadenas · colecciones · clases y objetos · métodos · constructores

Carrera	Ingeniería Informática — Facultad de Ciencias Puras, UATF
Asignatura	INF220 · Programación Orientada a Objetos y Estructuras de Datos
Unidad	U1 · Programación Orientada a Objetos
Lenguaje / entorno	Java (JDK Temurin 25 LTS) · Visual Studio Code
Modalidad	Trabajo individual
Ponderación	100 pts
Fecha de entrega	14 / 09 / 2026

Ejercicios propuestos

Resuelve los **10 ejercicios**. Cada uno indica los métodos o estructuras mínimas que debes utilizar.

EJ 01 Analizador de cadenas

Cadenas · Métodos predefinidos

Se necesita una herramienta que examine una frase ingresada por el usuario. Escriba un programa que reciba una cadena `texto` y, resolviendo cada punto **únicamente con métodos predefinidos de la clase `String`** (sin bucles), muestre por consola:

DEBE IMPLEMENTAR

- La longitud total de la cadena con `length()`.
- La cadena en mayúsculas y en minúsculas con `toUpperCase()` y `toLowerCase()`.
- La posición de la primera y la última aparición de la letra `a` con `indexOf()` e `lastIndexOf()`.
- La subcadena con los primeros 5 caracteres usando `substring()`.
- La cadena sin espacios al inicio ni al final con `trim()` o `strip()`.
- Si el texto contiene la palabra `Java`, empleando `contains()`.

EJ 02 Utilidades de texto propias

Cadenas · Métodos definidos por el programador

La clase `String` no ofrece todas las operaciones que un proyecto puede requerir. Cree una clase `UtilTexto` con los siguientes **métodos estáticos definidos por usted** (no se permite delegar la lógica en un método ya hecho de la librería):

DEBE IMPLEMENTAR

- `int contarVocales(String s)` — cuenta las vocales sin distinguir mayúsculas.
- `String invertir(String s)` — devuelve la cadena al revés recorriéndola manualmente (sin `StringBuilder.reverse()`).
- `boolean esPalindromo(String s)` — indica si se lee igual en ambos sentidos, ignorando mayúsculas y espacios.
- `int contarPalabras(String s)` — cantidad de palabras separadas por espacios.
- `String capitalizarPalabras(String s)` — pone en mayúscula la primera letra de cada palabra.
- Probar cada método desde `main` con al menos dos ejemplos distintos.

EJ 03 Registro de calificaciones

Listas · ArrayList · Métodos

Un docente lleva las notas de su curso. Con una `ArrayList<Double>`, escriba un programa que:

DEBE IMPLEMENTAR

- Solicite `n` y cargue `n` notas ingresadas por teclado en la lista.
- Calcule e imprima el promedio, la nota máxima y la mínima (puede apoyarse en `Collections.max` y `Collections.min`).
- Cuente cuántos estudiantes aprobaron, considerando aprobado con nota mayor o igual a 51.
- Construya una **segunda lista** solo con las notas aprobadas y la muestre.

EJEMPLO DE SALIDA ESPERADA

```
Notas: [45.0, 78.0, 51.0, 90.0, 33.0]
Promedio: 59.4 Máx: 90.0 Mín: 33.0
Aprobados: 3 -> [78.0, 51.0, 90.0]
```

EJ 04 Frecuencia de palabras

Cadenas · Diccionarios (Map)

Se desea analizar cuántas veces aparece cada palabra en un texto. Dada una cadena, sepárela en palabras con `split()` y construya un `HashMap<String,Integer>` que asocie cada palabra con su número de apariciones (ignorando mayúsculas y minúsculas). Luego:

DEBE IMPLEMENTAR

- Mostrar cada palabra junto con su frecuencia recorriendo el mapa con `entrySet()`.
- Indicar cuál es la palabra más repetida.
- Indicar cuántas palabras distintas contiene el texto.

EJEMPLO DE SALIDA ESPERADA

```
Texto: "el gato y el perro y el gato"
el=3 gato=2 y=2 perro=1
Más repetida: el | Palabras distintas: 4
```

EJ 05 Operaciones entre conjuntos

Conjuntos (Set)

Dos comisiones de estudiantes comparten algunos integrantes. A partir de dos `HashSet<String>` con nombres, calcule e imprima las operaciones clásicas de conjuntos:

DEBE IMPLEMENTAR

- La **unión** (todos los estudiantes) con `addAll()`.
- La **intersección** (quienes están en ambas comisiones) con `retainAll()`.
- La **diferencia** (quienes están solo en la primera) con `removeAll()`.
- A partir de una `List<String>` con nombres repetidos, obtener el conjunto de nombres únicos y reportar cuántos duplicados se eliminaron.
- Trabaje sobre **copias** de los conjuntos para no alterar los originales entre operación y operación.

EJ 06 Puntos en el plano (tuplas con record)

Tuplas → record · Listas

Java no dispone de tuplas nativas; para agrupar valores relacionados e inmutables usamos un `record`. Defina `record Punto(double x, double y)` y con él resuelva:

DEBE IMPLEMENTAR

- Crear una `List<Punto>` con al menos 5 puntos.
- Un método propio `double distancia(Punto a, Punto b)` que calcule la distancia euclidiana.
- Determinar cuál es el punto más lejano del origen `(0,0)`.
- Mostrar todos los puntos apoyándose en el `toString()` que el `record` genera automáticamente.
- Comentar en el código por qué un `record` es adecuado para representar una tupla inmutable.

EJ 07 Clase Libro

Clases y objetos · Constructores

Una biblioteca necesita representar sus libros. Cree la clase `Libro` con atributos privados `título`, `autor`, `año` y `disponible`, aplicando encapsulamiento.

DEBE IMPLEMENTAR

- Un **constructor** que reciba `título`, `autor` y `año`; el atributo `disponible` debe iniciar en `true`.
- Métodos `prestar()` y `devolver()` que modifiquen el estado de `disponible`.
- Un método `mostrarFicha()` que imprima todos los datos del libro y su disponibilidad.
- Desde `main`, crear al menos **tres objetos** `Libro`, prestar uno de ellos y mostrar la ficha de todos.

EJ 08 Sobrecarga de constructores (Rectángulo)

Constructores · `this()`

A veces un objeto puede crearse con distinta cantidad de datos. Cree la clase `Rectangulo` con atributos `base` y `altura`, y tres constructores sobrecargados:

DEBE IMPLEMENTAR

- Sin parámetros: `base` y `altura` valen 1.
- Con un parámetro (cuadrado): `base` y `altura` toman el mismo valor.
- Con dos parámetros: recibe `base` y `altura` por separado.
- Reutilizar código entre constructores mediante `this(...)`.
- Agregar los métodos `area()` y `perimetro()`.
- Desde `main`, crear un objeto con **cada** constructor y mostrar su área y perímetro.

EJ 09 Agenda de contactos

Clases y objetos · Listas · Métodos

Se quiere gestionar una agenda personal combinando una clase de datos con una colección. Cree la clase `Contacto` (constructor con `nombre`, `telefono` y `correo`) y una clase `Agenda` que contenga internamente una `ArrayList<Contacto>`.

DEBE IMPLEMENTAR

- `agregar(Contacto c)` — añade un contacto a la agenda.
- `buscarPorNombre(String nombre)` — devuelve el `Contacto` encontrado o `null`.
- `eliminar(String nombre)` — quita el contacto indicado si existe.
- `listar()` — muestra todos los contactos registrados.
- Desde `main`, agregar varios contactos, buscar uno, eliminar otro y listar el resultado final.

EJ 10 Integrador: analizador de texto

Cadenas · Map · Set · Clase · Constructor · Métodos

Ejercicio integrador. Construya la clase `AnalizadorTexto` cuyo **constructor** reciba el texto a analizar y calcule y guarde internamente: una `List<String>` con todas las palabras, un `Set<String>` con las palabras únicas y un `Map<String, Integer>` con la frecuencia de cada palabra.

La clase debe exponer los siguientes **métodos definidos por el programador**:

DEBE IMPLEMENTAR

- `int totalPalabras()` — total de palabras del texto.
- `int palabrasUnicas()` — cantidad de palabras distintas.
- `String palabraMasFrecuente()` — la palabra con mayor número de apariciones.
- `String resumen()` — arma una cadena con todas las estadísticas anteriores.
- Desde `main`, crear un objeto con un texto de ejemplo e imprimir su `resumen()`.